

Real-Time Financial Fraud Detection with Learned Hybrid Gating and Uncertainty-Aware Triage

Əskərov Nadir, Fətəliyev Akşin, Hüseynov Habil, Nəzərov Səbuhi

AI Academy, National Artificial Intelligence Center, Azerbaijan

Email: nadir.eskerov25@aiacademy.az, aqsin.fataliyev25@aiacademy.az,

habil.huseynov25@aiacademy.edu.az, sebuhi.nezerov25@aiacademy.az

Abstract—Financial fraud detection is challenging because fraudulent transactions are rare, behavior evolves over time, and operational systems must distinguish confident predictions from cases requiring human investigation. We present an end-to-end fraud detection and uncertainty-triage system that combines supervised and unsupervised deep learning. An FT-CAT Transformer produces supervised fraud probabilities from current and historical transaction information, while a denoising autoencoder provides an independent reconstruction-based anomaly signal. A learned gating network dynamically combines these signals with transaction-history activity and amount-intensity features. Split conformal prediction converts the fused fraud score into operational prediction sets for automatic approval, automatic blocking, or human review. On the chronological held-out test set, the FT-CAT model achieved a PR-AUC of 0.3412 and ROC-AUC of 0.8144. At a target miscoverage level of 0.01, the conformal system achieved 98.99% empirical marginal coverage with a 95% bootstrap confidence interval of 98.92–99.06%, while assigning 23.19% of transactions to human review (chronological test split; temporal shift violates strict exchangeability, so this is a measured result, not a production guarantee). CPU benchmarking measured a median batch-1 neural inference latency of 6.56 ms (P99 26.95 ms) and peak tested throughput of approximately 2,746 transactions per second at batch size 128; the 15 ms P99 service objective is met by the DAE scorer (0.71 ms eager P99 at batch size 1). The system additionally provides SHAP-based anomaly explanations, subgroup operational robustness auditing, a Streamlit analyst dashboard, FastAPI serving, and EXIR and ONNX model serialization. Repository: <https://github.com/sabuhi/DL-final-project>.

Index Terms—financial fraud detection, deep learning, transformer, anomaly detection, conformal prediction, uncertainty quantification

I. Introduction

Financial transaction systems must identify fraudulent behavior under severe class imbalance while operating under strict latency and reliability requirements. A useful fraud-detection product must therefore do more than produce a binary prediction. It should combine complementary evidence, expose uncertain cases to analysts, provide interpretable information about suspicious transactions, and remain deployable on modest hardware.

Purely supervised fraud classifiers can learn discriminative patterns from historical labels but may struggle when fraudulent behavior differs from previously observed examples. Conversely, unsupervised anomaly detectors can identify unusual transaction structure without relying exclusively on fraud labels, but unusual behavior is not necessarily fraudulent.

This motivates a hybrid architecture in which supervised and unsupervised signals contribute jointly to the final decision.

We develop an end-to-end fraud triage engine with four main components. First, an FT-CAT Transformer models the supervised probability of fraud using current transaction attributes and recent transaction history. Second, a denoising autoencoder trained on legitimate behavior produces a reconstruction-based anomaly score. Third, a learned gating network dynamically fuses the supervised probability, normalized anomaly score, history activity, and historical amount intensity. Finally, split conformal prediction transforms the fused score into prediction sets that support automatic approval, automatic blocking, and human review.

The system is designed as an Industry Product prototype rather than only an offline classifier. It includes a FastAPI inference service, a Streamlit analyst dashboard, SHAP-based explanations, model serialization, automated tests, CPU inference benchmarking, and retrospective subgroup robustness analysis.

The main contributions of this work are:

- an end-to-end hybrid fraud-detection architecture combining supervised FT-CAT prediction with unsupervised DAE anomaly detection;
- a learned gating network that dynamically combines model signals and causal transaction-history context instead of relying only on a fixed fusion coefficient;
- uncertainty-aware operational triage using split conformal prediction;
- explainability through SHAP analysis of the DAE anomaly component;
- deployment-oriented evaluation including CPU latency, throughput, model serialization, API serving, and an analyst-facing dashboard; and
- retrospective subgroup analysis for identifying variation in review workload and detection performance across available transaction attributes.

The remainder of this paper describes related work, the proposed architecture, the leakage-aware experimental protocol, empirical results, deployment characteristics, limitations, and conclusions.

II. Related Work

Fraud detection is commonly formulated as a highly imbalanced binary classification problem in which the minor-

ity fraud class is operationally more important than overall accuracy. Tree-based methods and neural classifiers are frequently used to model nonlinear interactions among transaction attributes, while precision-recall metrics are particularly informative under strong class imbalance.

Transformer architectures have increasingly been adapted to structured and tabular data [2]. Attention mechanisms [1] allow models to learn interactions among features and, when transaction histories are available, relationships between a current transaction and previous activity. Our supervised component follows this general direction by combining tabular transaction information with historical context.

Autoencoders provide a complementary approach to fraud detection. When trained primarily or exclusively on legitimate examples, reconstruction error can act as an anomaly score because transactions inconsistent with learned legitimate structure may reconstruct poorly. However, anomaly is not equivalent to fraud. Our system therefore treats reconstruction error as one signal rather than as the final decision.

Conformal prediction provides a model-agnostic framework for constructing prediction sets with finite-sample marginal coverage under exchangeability [4], [5]. Instead of forcing every transaction into a single predicted class, prediction sets can distinguish confident cases from ambiguous cases. We use this property to map uncertainty directly to an operational human-review queue.

III. Method

A. Problem Formulation

For each transaction x_i , the objective is to estimate whether its binary label $y_i \in \{0, 1\}$ corresponds to a legitimate or fraudulent transaction. The system produces a fused fraud probability

$$p_i = f_\theta(x_i, h_i), \quad (1)$$

where h_i represents causal historical information available before the current transaction.

Instead of using p_i only for hard binary classification, the final stage constructs a conformal prediction set

$$\Gamma_\alpha(x_i) \subseteq \{0, 1\}, \quad (2)$$

where α is the target miscoverage level.

B. FT-CAT Transformer

The supervised branch uses an FT-CAT Transformer to estimate fraud probability. The model consumes current transaction information together with a fixed recent history window. Continuous and categorical information is represented in a learned feature space, while attention mechanisms [1] model interactions between the current transaction and its historical context.

The supervised output is

$$P_{\text{FT}} = P(y = 1 \mid x, h). \quad (3)$$

This probability forms one of the primary inputs to the learned fusion stage.

C. Denoising Autoencoder

The unsupervised branch is a denoising autoencoder trained on legitimate transactions. Its encoder compresses an 800-dimensional numeric feature vector through

$$800 \rightarrow 256 \rightarrow 128 \rightarrow 32, \quad (4)$$

followed by a symmetric decoder

$$32 \rightarrow 128 \rightarrow 256 \rightarrow 800. \quad (5)$$

The network uses batch normalization and LeakyReLU activations. During training, feature corruption encourages the autoencoder to learn robust structure rather than simply copy the input.

For transaction x , the anomaly signal combines the squared reconstruction residual with an L_1 term that discourages a small number of large, concentrated residuals from dominating the score,

$$S_{\text{anomaly}}(x) = \sum_{j=1}^d (x_j - \hat{x}_j)^2 + \gamma_1 \sum_{j=1}^d |x_j - \hat{x}_j|. \quad (6)$$

The parameter $\gamma_1 = 0.4$ is fixed by configuration. The anomaly score is normalized by its 99.9th training-set percentile before entering the gating network.

D. Learned Hybrid Gating

A fixed weighted combination of anomaly and supervised scores assumes that the same mixture is appropriate for every transaction. We instead learn the fusion function.

The gating network receives four inputs,

$$z = [\tilde{S}_{\text{anomaly}}, P_{\text{FT}}, D_{\text{hist}}, A_{\text{hist}}], \quad (7)$$

where $\tilde{S}_{\text{anomaly}}$ is the normalized DAE anomaly score, D_{hist} is history density, and A_{hist} represents historical amount intensity.

History density is the fraction of active entries in the recent transaction window. Historical amount intensity is based on the logarithmically transformed mean absolute transaction amount among active historical rows. These quantities represent causal transaction-history activity and intensity rather than future-looking features.

The learned gating network maps these inputs to the final fraud probability,

$$P_{\text{gate}} = g_\phi(z). \quad (8)$$

E. Split Conformal Triage

The fused gate probability is passed to a split conformal predictor [4], [5]. Calibration examples are kept separate from the examples used to fit the upstream models and gating network.

For class y , a probability-based nonconformity score can be written as

$$s(x, y) = 1 - \hat{P}(y \mid x). \quad (9)$$

The calibration scores determine a finite-sample corrected quantile. Labels whose nonconformity scores fall below the calibrated threshold are included in the prediction set.

The resulting operational policy is

$$\Gamma_{\alpha}(x) = \begin{cases} \{0\}, & \text{Auto-Approve,} \\ \{1\}, & \text{Auto-Block,} \\ \{0, 1\}, & \text{Human Review,} \\ \emptyset, & \text{Human Review.} \end{cases} \quad (10)$$

Mapping the empty set to review is a defensive product decision: a transaction for which neither class is supported should not be silently approved or blocked.

F. Explainability

SHAP [3] is used to analyze the DAE anomaly component. A local waterfall visualization explains feature contributions for a selected high-anomaly transaction, while global analysis summarizes feature importance across sampled transactions.

These explanations apply specifically to the DAE reconstruction-anomaly component. They should not be interpreted as explanations of the complete learned-gate decision.

IV. Experimental Setup

A. Temporal Evaluation Protocol

Transactions are ordered chronologically to reduce temporal leakage. The original dataset is divided into approximately 70% training, 15% validation, and 15% final out-of-time testing blocks, while equal timestamps at split boundaries are kept together.

Additional subdivision is required because the learned gate, upstream model early stopping, and conformal calibration cannot safely reuse the same labels. The original training block is therefore subdivided into upstream model training and early-stopping subsets. The original validation period is reserved for gate development and conformal calibration. Gate development is itself separated into gate-training and gate early-stopping subsets.

This produces approximately 63% upstream-model training, 7% upstream-model early stopping, 6% gate training, 1.5% gate early stopping, 7.5% conformal calibration, and 15% final testing. Exact proportions can vary slightly because identical boundary timestamps remain grouped.

The final chronological test set contains 88,581 transactions and is not used for model fitting or conformal calibration.

B. Evaluation Metrics

Because fraud is highly imbalanced, accuracy alone is insufficient. Supervised and gating models are evaluated primarily using area under the precision-recall curve (PR-AUC), ROC-AUC, and true-positive rate at the 1% false-positive-rate operating point ($\max_{\text{fpr}}: 0.01$).

Conformal evaluation reports empirical marginal coverage, class-conditional coverage, review rate, singleton rate, review precision, reviewed-fraud capture, Brier score (mean squared probability error), and expected calibration error (ECE, binned confidence-accuracy gap).

Deployment evaluation reports latency percentiles and throughput over several batch sizes.

TABLE I
Classical baselines on the chronological development-validation set.

Model	PR-AUC	ROC-AUC
Logistic Regression (balanced)	0.4471	0.8609
Random Forest (200 trees, depth 16)	0.4698	0.8819
LightGBM (weighted)	0.5207	0.9088

For retrospective subgroup analysis, we report transaction count, observed fraud rate, review rate, recall, false-positive rate, and PR-AUC for available transaction and platform attributes. Subgroups with fewer than 30 transactions or fewer than 10 observed fraud cases are flagged as unreliable for subgroup-level interpretation.

C. Implementation and Reproducibility

The system is implemented in Python and PyTorch. Model and experiment configuration is centralized in configuration files, and fixed random seeds are used for reproducibility (`seed_everything(42)` for training runs; gate robustness evaluated at seeds 42, 123, and 2026; FT-CAT checkpoint records seed 42). Benchmarks were measured on a single-CPU container host with PyTorch CPU builds; the data split is strictly chronological (70/15/15 by `TransactionDT`) with scalars and vocabularies fitted on the training block only.

Automated regression tests cover the gating network, conformal predictor, evaluation utilities, SHAP integration, dashboard behavior, serialization, and artifact-isolation behavior.

The trained DAE, FT-CAT, and learned gate can be exported to PyTorch EXIR and ONNX. Export routines validate numerical parity against PyTorch inference.

V. Results

A. Classical Baselines

Logistic regression, random forest, and LightGBM baselines were trained on the upstream-model training split and evaluated on the chronological development-validation split (88,581 transactions). Table I reports validation PR-AUC and ROC-AUC.

The random forest uses a maximum depth of 16 and a minimum leaf size of 10: without a depth limit, 200 fully grown trees on the full training data produced a 550 MB checkpoint with multi-second inference stalls, while the constrained model compresses to 57 MB and scores the full validation split in 0.3 s on CPU. Final out-of-time test comparison against the deep models uses the same chronological held-out test set.

B. FT-CAT Performance

Table II reports the recorded final FT-CAT performance on the chronological held-out test set.

The recorded test PR-AUC of 0.3412 is numerically lower than the development-validation PR-AUC mean of 0.4395 from the architecture ablation, indicating that out-of-time generalization is harder than performance measured within the development period. This reinforces the importance of chronological evaluation for fraud detection.

TABLE II
FT-CAT performance on the held-out chronological test set.

Metric	Test Result
PR-AUC	0.3412
ROC-AUC	0.8144
TPR @ 1% FPR	0.2712
Loss	0.0117

C. Focal-Loss Sensitivity

A controlled focal-loss sensitivity experiment evaluated five focusing parameters, $\gamma \in \{0.5, 1.0, 2.0, 3.0, 5.0\}$, crossed with three class-balancing parameters, $\alpha \in \{0.1, 0.25, 0.5\}$, using seed 42 and an eight-epoch training budget. A weighted binary-cross-entropy configuration was included as a control.

To avoid test-set leakage in hyperparameter selection, configurations are compared using their best development-validation PR-AUC rather than their held-out test PR-AUC. Under this criterion, the strongest evaluated focal-loss cell was $\gamma = 0.5, \alpha = 0.5$, with validation PR-AUC 0.4325. Its held-out test PR-AUC was 0.3282. The weighted-BCE control achieved validation PR-AUC 0.4076 and test PR-AUC 0.3264.

The sweep indicates that focal-loss behavior is sensitive to both γ and α : increasingly aggressive focusing does not monotonically improve performance. In particular, large γ values can substantially reduce PR-AUC under the fixed training budget. Because the sweep uses one random seed and an eight-epoch budget, it should be interpreted as a sensitivity analysis rather than a definitive hyperparameter comparison.

D. Attention Analysis

To inspect how the FT-CAT checkpoint uses historical context, attention weights were summarized over 1,024 sampled transactions, balanced between 512 fraud and 512 legitimate examples. The analysis covers four attention heads and a five-position historical sequence.

Across the four heads, the most recent historical position receives substantial attention for both classes. Mean attention to the most recent slot is approximately 0.2875 for fraud examples and 0.3097 for legitimate examples. Several missingness-indicator features, including V261_is_nan, V256_is_nan, and V215_is_nan, appear among the features with the strongest observed history attention.

These results provide a descriptive view of how the trained network allocates attention across available historical context. Attention weights should not be interpreted as causal feature importance or as a complete explanation of the FT-CAT prediction.

E. Learned Gate Robustness

The learned gating model was evaluated across three fixed random seeds on the development-validation data. Results are shown in Table III.

The relatively small sample standard deviations (descriptive, $n = 3$; no significance test) indicate that the learned fusion model is not strongly dependent on a single random

TABLE III
Learned gate robustness across random seeds.

Seed	PR-AUC	ROC-AUC	TPR
42	0.3420	0.8373	0.2293
123	0.3293	0.8312	0.2348
2026	0.3432	0.8324	0.2431
Mean	0.3381	0.8336	0.2357
SD (sample, $n = 3$)	0.0077	0.0032	0.0070

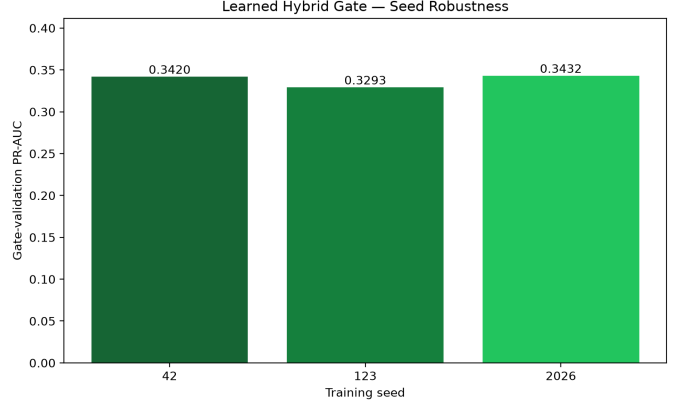


Fig. 1. Learned-gate performance across the three evaluated random seeds.

initialization. Seed 42 was used for the production checkpoint according to the project’s predetermined random seed rather than selected post hoc from the three results.

F. Conformal Triage

At $\alpha = 0.01$, split conformal prediction targets 99% finite-sample marginal coverage under exchangeability. Table IV reports performance on the chronological held-out test set.

The empirical marginal coverage of 98.99% is close to the nominal target. However, aggregate coverage masks substantial class asymmetry: legitimate coverage is 100%, while fraud coverage is approximately 70.94%. The review queue captures approximately 70.94% of fraudulent test transactions.

No transactions were assigned to the auto-block category at this conservative operating point. Under the chosen 99% coverage target, the system therefore behaves primarily as an automatic-approval plus human-review mechanism.

Figure 2 illustrates the operational trade-off between predictive coverage and analyst workload across the evaluated conformal operating points. At the selected $\alpha = 0.01$ operating point, the system achieves 98.99% empirical marginal coverage while routing 23.19% of test transactions to review.

Figure 3 provides a complementary view of the sensitivity of the conformal triage policy across evaluated operating points. The analysis emphasizes that the calibrated operating point affects both statistical coverage and downstream analyst workload; operational thresholds should therefore be chosen jointly with review-capacity requirements rather than from predictive performance alone.

TABLE IV
Conformal triage results at $\alpha = 0.01$.

Metric	Result
Target coverage	99.00%
Empirical coverage	98.99%
95% bootstrap CI	98.92–99.06%
Legitimate-class coverage	100.00%
Fraud-class coverage	70.94%
Human-review rate	23.19%
Singleton sets	76.81%
Auto-approve rate	76.81%
Auto-block rate	0.00%
Review precision	10.65%
Reviewed-fraud capture	70.94%
Brier score	0.0319
ECE	0.0121

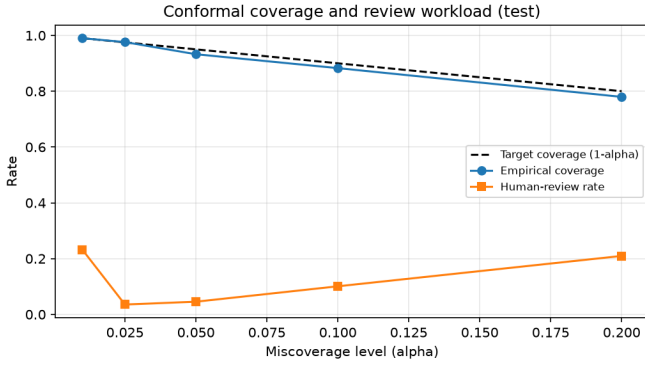


Fig. 2. Empirical conformal coverage and human-review workload across evaluated miscoverage levels.

G. Autoencoder Latent-Space Analysis

A t-SNE analysis was performed on 5,000 held-out transactions using the 32-dimensional DAE bottleneck representation. The resulting class-based silhouette score was 0.1046 (training log).

The visualization in Fig. 4 shows partial structural separation but substantial overlap between legitimate and fraudulent transactions. The weak but non-zero separation is consistent with the role of the DAE as an unsupervised anomaly detector rather than a standalone fraud classifier. The representation therefore provides complementary structural information rather than a clean fraud decision boundary.

H. SHAP Analysis

SHAP analysis was used to inspect the DAE anomaly component. Across independent global samples, the mean Spearman correlation of feature rankings was 0.8980, while mean top-feature Jaccard similarity was 0.4809.

Figure 5 shows the global SHAP importance profile for the DAE anomaly component. In the sampled analysis, `addr1` receives the strongest global anomaly attribution. The high mean Spearman rank correlation indicates that the broader ordering of feature importance is relatively stable across independent samples, although the lower mean top-feature

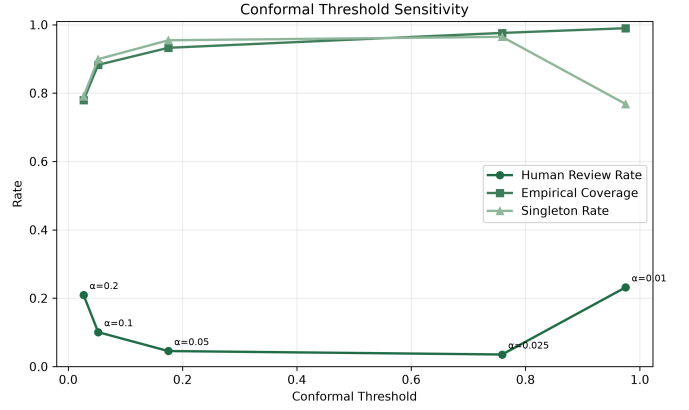


Fig. 3. Sensitivity of conformal triage behavior across evaluated operating points.

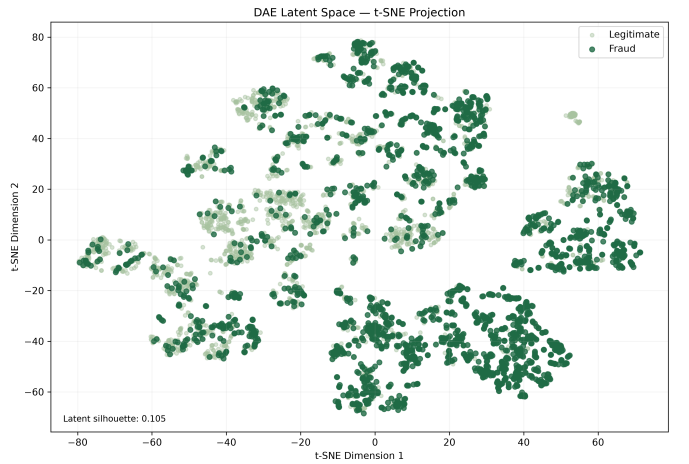


Fig. 4. t-SNE projection of the 32-dimensional DAE bottleneck representation.

Jaccard similarity shows that membership of the exact top-feature set is less stable.

Importantly, these attributions explain reconstruction anomaly rather than the final fused fraud probability. The prominence of `addr1` therefore should not be interpreted as evidence that it is the strongest predictor of fraud.

The consistency analysis in Fig. 6 complements the global importance plot by measuring how stable the DAE feature-attribution ranking is across independent samples. The high rank correlation indicates broad stability, while the more moderate Jaccard similarity demonstrates that the exact membership of the highest-ranked feature subset varies across samples.

Figure 7 provides a local example of the DAE explanation workflow. The waterfall visualization decomposes the anomaly-model output for an individual transaction into feature-level contributions. As with the global analysis, the visualization explains the DAE anomaly component only and should not be interpreted as a decomposition of the final learned-gate decision.

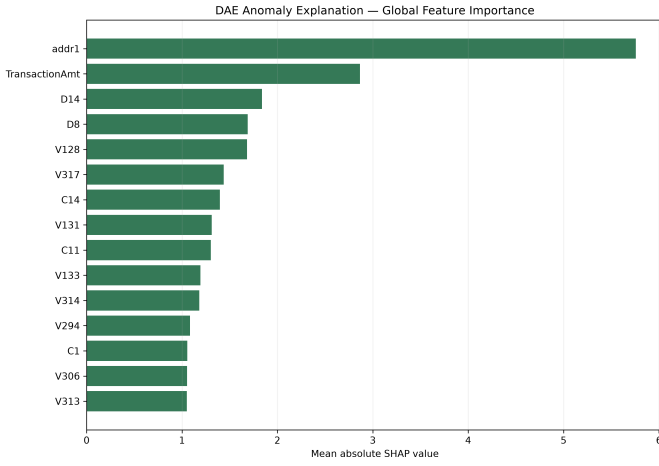


Fig. 5. Global SHAP feature importance for the DAE reconstruction-anomaly component.

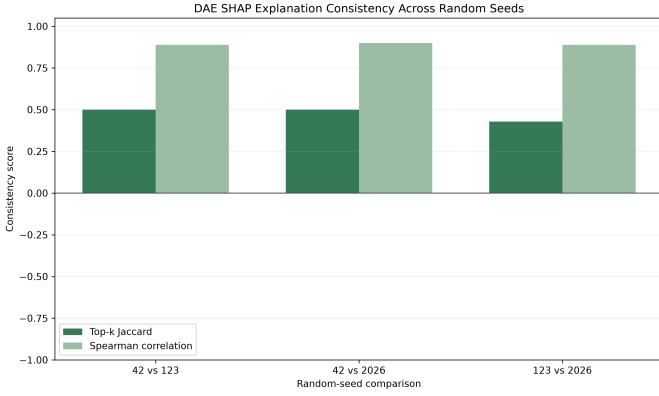


Fig. 6. Consistency of DAE SHAP feature attributions across independent samples.

I. Subgroup Operational Robustness Audit

As a retrospective robustness check, the final triage behavior was evaluated across available transaction subgroups defined by `DeviceType`, `card4`, `card6`, `ProductCD`, and `P_emaildomain`. For each subgroup, the audit reports transaction count, observed fraud rate, review rate, recall, false-positive rate, and PR-AUC. Groups with fewer than 30 transactions or fewer than 10 observed fraud cases are flagged as unreliable for subgroup-level interpretation.

The audit reveals substantial heterogeneity in operational behavior. For example, among the reported `ProductCD` groups, `ProductCD=W` has an approximately 14.2% review rate and 43% recall, whereas `ProductCD=S` has an approximately 89.6% review rate and 97% recall. Variation is also observed across `DeviceType`, `card4`, and `card6` groups. These differences demonstrate that aggregate coverage and review-rate statistics do not fully characterize how the triage policy behaves across transaction subpopulations.

These attributes describe transaction, payment, product, device, and platform characteristics rather than protected demo-

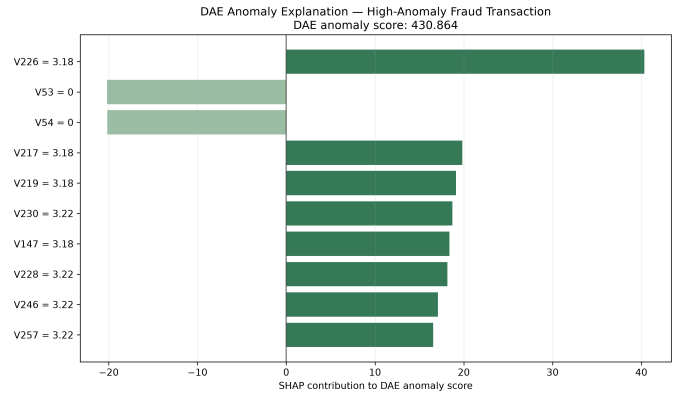


Fig. 7. Local SHAP waterfall explanation for a selected DAE anomaly example.

TABLE V
End-to-end neural CPU inference benchmark.

Batch	Median (ms)	P99 (ms)	Throughput
1	6.56	26.95	128 tx/s
32	14.90	24.08	2,092 tx/s
128	44.94	67.56	2,746 tx/s
512	189.73	290.84	2,693 tx/s

graphic characteristics. Accordingly, this analysis is an operational subgroup robustness audit and should not be interpreted as a demographic fairness assessment. It is intended to identify heterogeneous model behavior that warrants monitoring before production deployment.

J. CPU Inference Performance

The neural inference path was benchmarked on CPU using 10 warm-up iterations and 50 measured iterations per batch size. The benchmark includes the DAE, FT-CAT, transaction-history feature assembly, and learned gate. It excludes HTTP/Pydantic overhead, disk I/O, SHAP computation, and conformal decision logic.

Batch size 128 produced the highest tested throughput at approximately 2,746 transactions per second. At batch size 1, median neural inference latency was 6.56 ms. Component-level profiling at batch size 1 (median: FT-CAT 5.05 ms, DAE 0.42 ms, gate 0.12 ms) showed that FT-CAT dominates the neural compute path, whereas learned-gate inference itself is lightweight.

Figure 8 shows that throughput increases substantially with batching before reaching its highest tested value at batch size 128. Throughput rises from approximately 128 transactions per second at batch size 1 to 2,746 transactions per second at batch size 128, before remaining approximately flat at 2,693 transactions per second for batch size 512. Moderate batching therefore provides substantial computational efficiency, while increasing the batch size beyond 128 produced no additional throughput benefit in the tested CPU environment.

These measurements describe model-side neural inference and should not be interpreted as complete HTTP API latency.

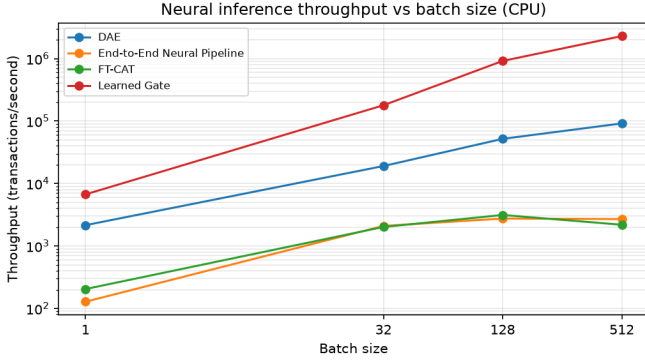


Fig. 8. Measured neural inference throughput across evaluated batch sizes on CPU.

All latency and throughput figures were measured on a single-CPU container host and are hardware-dependent: absolute timings will vary across CPUs, while the relative ordering of components and batch-size trends should reproduce.

At the component level, the servable DAE scorer was benchmarked separately with 200 timed iterations per cell on CPU across batch sizes 1–64. At batch size 1, P99 latency was 0.71 ms for PyTorch eager execution, 0.99 ms for the EXIR export, and 0.14 ms for ONNX, all well within the 15 ms service objective; DAE-scorer P99 remained below 1 ms at every tested batch size.

K. Deployment and Serialization

The trained DAE (485,440 trainable parameters plus 513 non-trainable batch-normalization buffers, 1.95 MB checkpoint) was serialized to both PyTorch EXIR (2.00 MB) and ONNX (1.96 MB across the model and data files) representations. Export-time parity checks compare serialized outputs with their PyTorch reference implementations.

The trained FT-CAT (94,305 parameters) and learned-gate (225 parameters) checkpoints have been exported through the same pipeline to `models/serialized/` in EXIR and ONNX formats. Serialized parity reporting for these models follows the DAE procedure.

The containerized FastAPI scoring service (single CPU worker) ships as an approximately 2 GB uncompressed inference image (roughly 450 MB compressed transfer size), dominated by the PyTorch CPU runtime and model weights.

The product additionally exposes a FastAPI scoring interface and a Streamlit analyst dashboard. The dashboard surfaces live prediction, batch analysis, model insights, conformal triage, explainability, and artifact-readiness information.

VI. Discussion and Limitations

The experiments demonstrate the value of treating fraud detection as a triage problem rather than forcing every transaction into an automatic binary decision. The supervised FT-CAT branch provides discriminative fraud evidence, while the DAE contributes a complementary anomaly signal. Learned gating

provides a flexible mechanism for combining these signals with transaction history.

The conformal results expose an important operational trade-off. The system achieves 98.99% empirical marginal coverage on the chronological test set at the 99% target, but approximately 23.19% of transactions require human review. At this conservative operating point, no test transactions are automatically blocked. Therefore, the current configuration is better characterized as a high-coverage approval-and-review system than as a fully automated fraud-blocking system.

A second limitation is class-conditional behavior. Marginal coverage is close to the nominal target, but fraud-class coverage is substantially lower than legitimate-class coverage. Marginal split conformal prediction does not guarantee equal class-conditional coverage.

The subgroup audit further demonstrates that aggregate performance can conceal substantial operational heterogeneity. Review workload and fraud recall differ considerably across some product, device, and card-related subgroups. These differences do not establish demographic unfairness because the audited variables are not protected demographic attributes, but they show that a single aggregate operating point can affect transaction subpopulations very differently. Production deployment would therefore require continued subgroup monitoring and investigation of the mechanisms producing these disparities.

A third limitation concerns the assumptions behind conformal prediction. Finite-sample marginal coverage applies under exchangeability between calibration and future examples. Financial transaction data are chronological and may undergo temporal or adversarial distribution shift. Consequently, 98.99% should be reported as observed chronological test coverage, not as an unconditional guarantee for future production traffic.

A fourth limitation was identified and corrected during development. An early version of the automated test suite for the conformal evaluation script wrote its output artifacts to the same path used for production evaluation rather than an isolated temporary directory. Because tests were run locally during development, this could silently regenerate the serving threshold artifact from a small synthetic test fixture instead of the real calibration split.

The issue was caught, the test harness was corrected to redirect all artifact writes to an isolated path, and the affected artifacts were regenerated from the real calibration data. It is reported here for transparency, and because it illustrates a general risk in ML pipelines: test code that exercises production code paths without isolating side effects can silently corrupt deployed artifacts without raising any error.

A fifth limitation concerns model complexity versus measured benefit. An ablation comparing the fused FT-CAT Transformer (94,305 parameters) against a self-attention-only variant and a flat MLP control, trained under an identical protocol across three random seeds, found that the flat MLP control achieved the highest mean test PR-AUC (0.3698) of the three, exceeding both the self-attention-only variant

(0.3376) and the full FT-CAT model (0.3380). Capacity and training budget were held fixed rather than independently tuned per variant, so this does not prove categorical feature interactions are unhelpful on this dataset. It does mean that the transformer’s added architectural complexity is not currently justified by test-set performance and is reported rather than left implicit.

A related design limitation is the construction of transaction histories using `card1` together with `addr1` as a pseudo-entity key. These are masked dataset attributes rather than a verified persistent customer, account, or card identifier. Consequently, the resulting sequences may combine transactions that do not belong to a single real-world entity. History-based features should therefore be interpreted as pseudo-entity behavioral context rather than true per-card transaction histories.

The focal-loss sensitivity experiment is also intentionally limited. It uses a single random seed and a fixed eight-epoch budget, so the observed differences across γ and α should not be interpreted as a fully tuned comparison. The experiment is used to characterize sensitivity and to enforce validation-based rather than test-based configuration selection.

The attention analysis is descriptive rather than causal. Attention weights show where the trained FT-CAT allocates representation weight across historical positions but do not establish that highly attended features caused the model’s prediction.

The SHAP implementation is similarly scoped. Current explanations describe the DAE anomaly component and do not provide a complete attribution of the final learned-gate decision. Extending explainability to the complete fused system is an important direction for future work.

The fixed-coefficient and learned-gating comparison also requires careful interpretation. The learned gate incorporates transaction-history activity and amount-intensity features in addition to the DAE and FT-CAT signals, whereas a simple fixed-weight fusion uses only the two model scores. Improvements from the learned system therefore cannot be attributed exclusively to nonlinear fusion; the additional causal history context is also part of the learned-gate design.

Finally, the latency benchmark measures the neural computation path rather than complete request-to-response service latency. Production deployment would require additional benchmarking of network overhead, request validation, concurrency, monitoring, and data retrieval.

VII. Conclusion

We presented an end-to-end financial fraud detection and uncertainty-triage engine combining an FT-CAT Transformer, a denoising autoencoder, learned hybrid gating, and split conformal prediction.

The system achieved 98.99% empirical marginal coverage at a nominal 99% target on the chronological held-out test set while routing 23.19% of transactions to human review. The results demonstrate the practical role of uncertainty-aware triage: rather than forcing every prediction into an automated

action, the system explicitly identifies cases requiring analyst intervention.

The final evaluation additionally examines focal-loss sensitivity, attention behavior, learned-gate seed robustness, DAE latent structure, SHAP attribution consistency, and subgroup operational heterogeneity. Together, these analyses provide a broader view of model robustness and limitations than aggregate predictive metrics alone.

The deployment prototype supports FastAPI serving, an analyst-facing Streamlit dashboard, SHAP-based anomaly explanations, and EXIR and ONNX serialization. CPU benchmarking demonstrated a median batch-1 neural inference latency of 6.56 ms (P99 26.95 ms; the 15 ms P99 objective is met by the DAE scorer) and peak tested throughput of approximately 2,746 transactions per second.

Future work should investigate class-conditional conformal calibration, distribution-shift and subgroup monitoring, fused-model explanations, alternative sequence-identity definitions, and operating points that support safe automatic blocking.

Tools and Acknowledgements

We used AI-assisted tools, including ChatGPT, to support code debugging, software review, test development, and wording refinement during the project. All experiments, reported metrics, implementation decisions, and final materials were reviewed and validated by the authors.

We acknowledge the AI Academy and National Artificial Intelligence Center for the Deep Learning Cohort I course environment and project framework. We also acknowledge Vesta Corporation and the IEEE Computational Intelligence Society for the IEEE-CIS Fraud Detection dataset [6] used to train and evaluate the system.

References

- [1] A. Vaswani et al., “Attention is all you need,” in *Proc. Advances in Neural Information Processing Systems*, 2017.
- [2] Y. Gorishniy, I. Rubachev, V. Khrulkov, and A. Babenko, “Revisiting deep learning models for tabular data,” in *Proc. Advances in Neural Information Processing Systems*, 2021.
- [3] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Proc. Advances in Neural Information Processing Systems*, 2017.
- [4] V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic Learning in a Random World*. New York, NY, USA: Springer, 2005.
- [5] A. N. Angelopoulos and S. Bates, “Conformal prediction: A gentle introduction,” *Foundations and Trends in Machine Learning*, vol. 16, no. 4, pp. 494–591, 2023.
- [6] IEEE Computational Intelligence Society and Vesta Corporation, “IEEE-CIS Fraud Detection,” Kaggle competition, 2019. [Online]. Available: <https://www.kaggle.com/competitions/ieee-fraud-detection>